

CELEBAL EXCELLENCE INTERNSHIP
PROGRAM

DATA ENGINEERING

WEEK - 5 ASSIGNMENT REPORT

SPARK QUESTIONS

SUBMITTED BY:-

ANUSHKA SHARMA

(JKLU)

➤ OBJECTIVE

The objective of this assignment is to understand the fundamentals of Apache Spark and perform data cleaning, filtering, transformation, grouping, and aggregation using Spark DataFrames. The assignment also covers concepts such as MapReduce, in-memory processing, DataFrame immutability, shuffle operations, and building a simple data processing pipeline.

➤ DATASET DESCRIPTION

A sample CSV dataset containing **500 records** was used for this assignment. The dataset includes customer and sales information such as user ID, transaction date, city, region, product category, price, sale amount, subscription type, timestamps, and store ID. The dataset also contains duplicate records and null values to demonstrate different data cleaning operations.

Spark Setup

✓ Step 1: Import Required Libraries

Import the required Spark libraries and create a Spark Session.

```
# Import required Spark libraries

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

[1]


```

# Display dataset schema
df.printSchema()

```

[6]

```

...  root
      |-- user_id: string (nullable = true)
      |-- transaction_date: date (nullable = true)
      |-- product_category: string (nullable = true)
      |-- sale_amount: double (nullable = true)
      |-- region: string (nullable = true)
      |-- city: string (nullable = true)
      |-- age: integer (nullable = true)
      |-- subscription: string (nullable = true)
      |-- status: string (nullable = true)
      |-- price: double (nullable = true)
      |-- store_id: string (nullable = true)
      |-- raw_timestamp: string (nullable = true)
      |-- email: string (nullable = true)
      |-- username: string (nullable = true)

```

```

# Count total number of rows
df.count()

```

[7]

```

...  500

```

```

# Display column names and total columns
print("Columns:" , df.columns)
print("Number of columns:", len(df.columns))

```

[8]

```

...  Columns: ['user_id', 'transaction_date', 'product_category', 'sale_amount', 'region', 'city', 'age', 'subscription', 'status', 'price', 'store_id', 'raw_timestamp', 'email', 'username']
      Number of columns: 14

```

The dataset was loaded successfully into a Spark DataFrame. The first few records, schema, and column names were checked to understand the structure of the data before performing any operations.

➤ ASSIGNMENT QUESTIONS

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer:

Traditional **MapReduce** stores intermediate results on disk after every Map and Reduce stage. This causes more disk read/write operations, making processing slower, especially for large datasets.

Apache Spark is preferred because it processes data mainly in memory (RAM), reducing disk access and improving performance. It also provides easy-to-use APIs like DataFrames and supports SQL, machine learning, graph processing, and streaming in a single framework.

Key Limitations of MapReduce

- Uses disk for intermediate data, making it slower.
- Requires multiple jobs for complex workflows.
- Not suitable for iterative tasks like machine learning.
- Higher execution time because of frequent disk I/O.
- More complex code compared to Spark.

Why Spark is Better

- Processes data in memory, making it much faster.
- Supports DataFrames for easier programming.
- Suitable for batch, streaming, SQL, and machine learning.
- Reduces execution time by minimizing disk operations.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Spark stores intermediate data in **memory (RAM)** instead of writing it to disk after every operation. In machine learning, the same dataset is processed many times during training. Since Spark keeps the data in memory, it avoids repeated disk reads and writes, making execution much faster than disk-based systems like MapReduce.

For example, if an algorithm needs to process the same data 20 times, Spark can reuse the data already stored in memory instead of loading it from disk each time.

Q3. Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Q3 - Remove Duplicate Records

Remove duplicate rows based on the `user_id` and `transaction_date` columns.

```
# Remove duplicates based on user_id and transaction_date

df_no_duplicates = df.dropDuplicates(["user_id", "transaction_date"])

print("Rows before removing duplicates:", df.count())
print("Rows after removing duplicates:", df_no_duplicates.count())

df_no_duplicates.show(5)
```

[9]

... Rows before removing duplicates: 500

Rows after removing duplicates: 480

user_id	transaction_date	product_category	sale_amount	region	city	age	subscription	status	price
U1001	2026-06-27	Groceries	3453.62	East	Jaipur	37	Premium	Cancelled	2558.76
U1002	2026-02-18	Electronics	4759.89	West	Pune	47	Premium	Completed	113.78
U1003	2026-03-29	Clothing	1567.15	South	Mumbai	31	Basic	Cancelled	2052.65
U1004	2026-05-21	Clothing	3129.7	South	Jaipur	38	Premium	Pending	461.23
U1005	2026-05-18	Furniture	2018.67	South	Jaipur	34	Basic	Completed	NULL

only showing top 5 rows

[10]

```
# Use the updated DataFrame
df = df_no_duplicates
```

Insight: The dataset had 500 records initially. After removing duplicate records based on `user_id` and `transaction_date`, 480 unique records remained.

Q4. Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Q4 - Filter and Aggregate Data

Filter records where the region is **West** and calculate the average sale amount for each product category.

```
# Filter records where region is 'West'
west_sales = df.filter(col("region") == "West")

# Group by product category and calculate average sale amount
avg_sales = west_sales.groupBy("product_category").agg(avg("sale_amount").alias("average_sale_amount"))

avg_sales.show()
```

[11]

```
... +-----+-----+
|product_category|average_sale_amount|
+-----+-----+
|      Sports   | 2820.9434615384607 |
|   Groceries   | 3186.9576666666662 |
| Electronics   | 2322.0270000000005 |
|   Clothing    | 2234.2345454545457 |
|   Furniture   | 2373.1550000000007 |
+-----+-----+
```

Insight: The data was filtered for the West region and grouped by product category. The output shows the average sale amount for each category, helping compare sales performance.

Q5. What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

- **`.na.drop()`** removes rows that contain null values.
- **`.na.fill()`** replaces null values with a specified value instead of removing the rows.

Q5 - Handle Missing Values

Demonstrate the use of `.na.drop()` and `.na.fill()` to handle missing values.

```
# Removing rows with null values
df.na.drop().show(5)
```

[12]

```
... +-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|transaction_date|product_category|sale_amount|region|city|age|subscription|status|price|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|U1001|2026-06-27|Groceries|3453.62|East|Jaipur|37|Premium|Cancelled|2558.76|
|U1002|2026-02-18|Electronics|4759.89|West|Pune|47|Premium|Completed|113.78|
|U1003|2026-03-29|Clothing|1567.15|South|Mumbai|31|Basic|Cancelled|2052.65|
|U1004|2026-05-21|Clothing|3129.7|South|Jaipur|38|Premium|Pending|461.23|
|U1007|2026-03-29|Clothing|382.08|West|Delhi|46|Basic|Cancelled|2339.94|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
# Fill null values in the status column
df_filled = df.na.fill({"status": "Unknown"})
df_filled.select("status").show(10)
```

[13]

```
... +-----+
|status|
+-----+
|Cancelled|
|Completed|
|Cancelled|
|Pending|
|Completed|
|Unknown|
|Cancelled|
|Cancelled|
|Cancelled|
|Cancelled|
+-----+
only showing top 10 rows
```

```
# Use the updated DataFrame
df = df_filled
```

Insight: In this question, both `.na.drop()` and `.na.fill()` were demonstrated. For further processing, `.na.fill()` was used to replace null values in the status column with "Unknown".

Q6. Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

✓ Q6 - Count Records by City

Find cities having more than 100 records using `groupBy()` and `count()`.

```
# Count records for each city having more than 100 records
city_count = df.groupBy("city").count().filter(col("count") > 100)
city_count.show()
```

[15]

```
...  +-----+-----+
      |  city|count|
      +-----+-----+
      | Delhi|  129|
      |Jaipur|  220|
      +-----+-----+
```

Insight: The data was grouped by city and only cities with more than 100 records were displayed. In this dataset, Jaipur and Delhi met this condition.

Q7. How does the immutability of Spark DataFrames affect how you perform data cleaning steps like dropping columns or renaming them?

Answer:

Spark DataFrames are **immutable**, which means they **cannot be changed after they are created**.

When we perform operations like dropping a column, renaming a column, or filtering data, Spark does not modify the original DataFrame. Instead, it creates a **new DataFrame** containing the changes.

For example: `newdf = df.drop("price")`

Here, `df` remains unchanged, and `newdf` contains the updated data.

This makes data processing safer because the original data is always preserved unless we explicitly assign the new DataFrame back to the same variable.

Q8. Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
Q8 - Filter Premium Users

Filter users whose age is between 18 and 30 and whose subscription type is Premium.

# Filter Premium users between age 18 and 30
premium_users = df.filter((col("age").between(18, 30)) & (col("subscription") == "Premium"))
premium_users.show(5)
```

[16]

user_id	transaction_date	product_category	sale_amount	region	city	age	subscription	status	price	store
U1016	2026-01-17	Clothing	3466.87	South	Jaipur	27	Premium	Completed	1313.87	
U1017	2026-01-30	Groceries	723.13	North	Jaipur	27	Premium	Cancelled	1074.44	
U1019	2026-05-25	Furniture	866.55	West	Mumbai	20	Premium	Completed	1926.24	
U1025	2026-01-02	Groceries	4472.67	West	Jaipur	21	Premium	Pending	2688.13	
U1032	2026-01-14	Groceries	1043.77	South	Mumbai	22	Premium	Cancelled	1621.13	

only showing top 5 rows

Insight: This filtering helps identify young users with a Premium subscription, making it easier to analyze this specific customer group.

Q9. When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Answer:

Handling null values before performing calculations helps produce more accurate results.

If important values are missing, calculations like **sum()**, **avg()**, or **min()** may give incorrect or incomplete results. Replacing null values with appropriate values or removing invalid records improves the quality of the analysis.

For example, if the price column contains null values, filling them with **0** or removing those records (depending on the situation) ensures that calculations are performed on clean data.

Q10. Write the code to revise a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

Q10 - Modify Schema

Convert the `raw_timestamp` column to `TimestampType` and rename it to `event_time`.

```
# Convert raw_timestamp to TimestampType
from pyspark.sql.types import TimestampType

df = df.withColumn("event_time", col("raw_timestamp").cast(TimestampType())).drop("raw_timestamp")
df.printSchema()
```

[17]

```
... root
|-- user_id: string (nullable = true)
|-- transaction_date: date (nullable = true)
|-- product_category: string (nullable = true)
|-- sale_amount: double (nullable = true)
|-- region: string (nullable = true)
|-- city: string (nullable = true)
|-- age: integer (nullable = true)
|-- subscription: string (nullable = true)
|-- status: string (nullable = false)
|-- price: double (nullable = true)
|-- store_id: string (nullable = true)
|-- email: string (nullable = true)
|-- username: string (nullable = true)
|-- event_time: timestamp (nullable = true)
```

Insight: The `raw_timestamp` column was converted to `TimestampType` and renamed to `event_time`, making the date and time values easier to use.

Q11. Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer:

A **shuffle** is the process of moving data between different partitions or worker nodes during certain Spark operations.

When we use operations like **groupBy()**, Spark needs to bring together records with the same key. Since these records may be stored in different partitions, Spark transfers the required data across the cluster. This data movement is called **shuffle**.

It is called a **wide transformation** because data is moved between multiple partitions instead of being processed within the same partition. Shuffle can increase execution time because moving data across partitions is more expensive than processing it locally.

Q12. Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

```
Q12 - Clean Invalid Records

Remove records where the email is null or the username is empty.

# Remove rows with null email or empty username
clean_df = df.filter((col("email").isNull()) & (col("username") != ""))

print(df.count())
print(clean_df.count())

[18]
... 480
      451

# Use the updated DataFrame
df = clean_df
```

Insight: Records with a null email or an empty username were removed. The total number of records reduced from 480 to 451 after cleaning.

Q13. How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

✓ Q13 - Aggregate Statistics

Calculate the minimum, maximum, and average values of the `price` column.

```
# Calculate minimum, maximum and average price
df.agg(
    min("price").alias("Minimum Price"),
    max("price").alias("Maximum Price"),
    avg("price").alias("Average Price")
).show()
```

[20]

```
...  +-----+-----+-----+
      |Minimum Price|Maximum Price|   Average Price|
      +-----+-----+-----+
      |          52.18|          2993.6|1539.1979118329464|
      +-----+-----+-----+
```

Insight: The `.agg()` function calculated the minimum, maximum, and average price in a single operation. These values provide a quick summary of the price distribution in the dataset.

Q14. In the context of cleaning a dataset, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Answer:

When `inferSchema=true` is used, Spark automatically detects the data type of each column based on the data it reads.

If a date column contains inconsistent formats, Spark may:

- Detect the wrong data type.

Insight: Since duplicate records were already removed, the pipeline focused on filling missing prices and calculating the total revenue for each store. The output can be used to compare the revenue generated by different stores.

➤ **CONCLUSION**

In this assignment, Apache Spark DataFrames were used to clean, transform, filter, and analyze a CSV dataset. Operations such as duplicate removal, null value handling, filtering, aggregation, schema modification, and building a simple data processing pipeline were successfully performed. The assignment also helped in understanding important Spark concepts such as in-memory processing, DataFrame immutability, and shuffle operations.